

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание 1

Выполнила:

Савченко Анастасия

Группа К3341, Поток БР 1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Описать функциональные и нефункциональные требования к бэкенд-приложению для бронирования столиков в ресторанах, выбрать формат реализации API, спроектировать все эндпоинты в формате OpenAPI-схемы с учётом ER-диаграммы из ДЗ1, описать тела запросов и ответов, продумать возможные ошибки.

Ход работы

1. Описание функциональных требований

Приложение должно обеспечивать следующий функционал:

Регистрация и аутентификация пользователей:

- Регистрация нового пользователя с указанием электронной почты, пароля, имени и опционально номера телефона
- Вход в систему по электронной почте и паролю с получением JWT-токенов (access и refresh)
- Обновление токенов доступа через refresh-токен без повторного ввода пароля

Управление профилем:

- Просмотр профиля текущего пользователя (почта, имя, телефон, даты создания и обновления)
- Частичное редактирование профиля: изменение имени и номера телефона
- Просмотр истории своих бронирований с информацией о ресторане, столике, дате, времени и статусе

Поиск и просмотр ресторанов:

- Получение списка ресторанов с возможностью фильтрации по городу, типу кухни и ценовой категории
- Полнотекстовый поиск ресторанов по названию

- Пагинация списка с указанием общего количества записей и страниц

- Получение подробной информации о конкретном ресторане: описание, адрес, координаты, список кухонь (через ассоциативную таблицу RestaurantCuisine), массив фотографий, средний рейтинг и количество отзывов

Справочник кухонь:

- Получение полного списка типов кухонь для использования в фильтрах

Меню ресторана:

- Просмотр меню конкретного ресторана с фильтрацией по категориям блюд

- Добавление новых блюд администратором ресторана

- Частичное редактирование информации о блюде

- Удаление блюда из меню

Управление столиками:

- Просмотр списка всех столиков ресторана с их характеристиками

- Проверка доступности столиков на конкретную дату и временной интервал с учётом количества гостей

Бронирования:

- Создание нового бронирования с указанием ресторана, столика, даты, времени начала и окончания, количества гостей и дополнительных пожеланий

- Просмотр деталей конкретного бронирования

- Отмена бронирования с проверкой прав доступа и текущего статуса

Отзывы:

- Просмотр отзывов о ресторане с пагинацией

- Создание отзыва с обязательной оценкой от 1 до 5 и опциональным текстовым комментарием

- Редактирование собственного отзыва

- Удаление собственного отзыва

Фотографии ресторана:

- Просмотр всех фотографий ресторана

- Добавление новых фотографий администратором

- Удаление фотографий

2. Нефункциональные требования

Формат обмена данными: все запросы и ответы используют формат JSON

Аутентификация и авторизация: доступ к защищённым эндпоинтам осуществляется через JWT-токены в заголовке Authorization

Валидация входных данных: все входящие данные проверяются на корректность, невалидные запросы отклоняются

Единый формат ошибок: все ошибки возвращаются в стандартизированной структуре, содержащей код ошибки, текстовое описание и HTTP-статус

Пагинация: для эндпоинтов, возвращающих списки, реализована постраничная навигация

Безопасность: пароли хранятся в хэшированном виде, все запросы к базе данных параметризованы через ORM

Логирование: основные операции записываются в лог

CORS: настроена поддержка кросс-доменных запросов

Версионирование: все эндпоинты используют префикс /api/v1

3. Выбор формата реализации API

Для данного приложения выбран архитектурный стиль REST API.
Обоснование:

Чётко выделены ресурсы: пользователи, рестораны, кухни, блюда, столики, бронирования, отзывы, фотографии

Каждый ресурс имеет уникальный URL-идентификатор

Используются стандартные HTTP-методы: GET, POST, PATCH, DELETE

Структура эндпоинтов интуитивно понятна и соответствует принципам RESTful

API легко документируется в формате OpenAPI и тестируется через Postman

BFF-подход избыточен, так как предполагается один тип клиентского приложения

4. Структура API

API разделено на девять логических групп эндпоинтов:

Auth — регистрация, вход и обновление токенов:

- POST /api/v1/auth/register — регистрация нового пользователя
- POST /api/v1/auth/login — вход в систему
- POST /api/v1/auth/refresh — обновление токенов

Users — управление профилем:

- GET /api/v1/users/me — получение профиля
- PATCH /api/v1/users/me — редактирование профиля

Restaurants — рестораны:

— GET /api/v1/restaurants — список ресторанов с фильтрацией и пагинацией

— GET /api/v1/restaurants/{restaurant_id} — детальная информация о ресторане

Cuisines — справочник кухонь:

- GET /api/v1/cuisines — список всех кухонь

Dishes — меню:

- GET /api/v1/restaurants/{restaurant_id}/dishes — меню ресторана

- POST /api/v1/restaurants/{restaurant_id}/dishes — добавить блюдо
- PATCH /api/v1/dishes/{dish_id} — редактировать блюдо
- DELETE /api/v1/dishes/{dish_id} — удалить блюдо

Tables — столики:

- GET /api/v1/restaurants/{restaurant_id}/tables — список столиков
- GET /api/v1/restaurants/{restaurant_id}/tables/available —

доступные столики

Reservations — бронирования:

- GET /api/v1/reservations — бронирования пользователя
- POST /api/v1/reservations — создать бронирование
- GET /api/v1/reservations/{reservation_id} — детали бронирования
- PATCH /api/v1/reservations/{reservation_id}/cancel — отменить

бронирование

Reviews — отзывы:

— GET /api/v1/restaurants/{restaurant_id}/reviews — отзывы о ресторане

- POST /api/v1/restaurants/{restaurant_id}/reviews — оставить отзыв
- PATCH /api/v1/reviews/{review_id} — редактировать отзыв
- DELETE /api/v1/reviews/{review_id} — удалить отзыв

Photos — фотографии:

- GET /api/v1/restaurants/{restaurant_id}/photos — фото ресторана
- POST /api/v1/restaurants/{restaurant_id}/photos — добавить фото
- DELETE /api/v1/photos/{photo_id} — удалить фото

5. Описание запросов и ответов

Регистрация (POST /auth/register): принимает email, пароль, имя и опционально телефон. Возвращает профиль пользователя с идентификатором, почтой, именем, телефоном и датой создания. Ошибки: 400 — email занят, 422 — невалидные данные.

Вход (POST /auth/login): принимает email и пароль. Возвращает access-токен, refresh-токен и время жизни. Ошибка 401 при неверных данных.

Профиль (GET /users/me): требует авторизацию. Возвращает полную информацию о пользователе, включая даты создания и обновления.

Редактирование профиля (PATCH /users/me): принимает name и phone_num (оба опциональны). Возвращает обновлённый профиль.

Список ресторанов (GET /restaurants): поддерживает фильтры: city_name, cuisine_id, price_tier, search. Пагинация через page и limit. Ответ содержит массив ресторанов (id, название, город, цена, фото, рейтинг, кухни) и объект пагинации.

Детали ресторана (GET /restaurants/{restaurant_id}): возвращает описание, адрес, координаты, массив кухонь, массив фотографий, средний рейтинг, количество отзывов. Кухни получаются через JOIN ассоциативной таблицы RestaurantCuisine. Ошибка 404 при отсутствии ресторана.

Список кухонь (GET /cuisines): возвращает массив всех кухонь с id и названием.

Меню (GET /restaurants/{restaurant_id}/dishes): поддерживает фильтр по категории. Возвращает блюда с названием, описанием, ценой, категорией, доступностью и изображением.

Добавление блюда (POST /restaurants/{restaurant_id}/dishes): принимает название, описание, цену, категорию, доступность и URL изображения. Требуется прав администратора. Возвращает созданное блюдо.

Редактирование блюда (PATCH /dishes/{dish_id}): позволяет частично обновить любое поле.

Столики (GET /restaurants/{restaurant_id}/tables): возвращает все столики с номерами, вместимостью и зоной.

Доступные столики (GET /restaurants/{restaurant_id}/tables/available): обязательные параметры: дата, время начала и окончания. Опционально: количество гостей. Возвращает свободные столики.

Бронирования пользователя (GET /reservations): требует авторизации. Возвращает список бронирований с рестораном, столиком, датой, временем и статусом.

Создание бронирования (POST /reservations): принимает restaurant_id, table_id, дату, время, количество гостей и опционально примечания. Ошибка 409 при конфликте по времени.

Отмена бронирования (PATCH /reservations/{reservation_id}/cancel): меняет статус на cancelled. Ошибка 403 при попытке отменить чужое бронирование, 409 если уже отменено или завершено.

Отзывы (GET /restaurants/{restaurant_id}/reviews): пагинированный список с автором, оценкой, комментарием и датой.

Создание отзыва (POST /restaurants/{restaurant_id}/reviews): обязательна оценка от 1 до 5, комментарий опционален.

Редактирование отзыва (PATCH /reviews/{review_id}): изменить можно только свой отзыв.

6. Ассоциативная таблица RestaurantCuisine

Связь «многие-ко-многим» между ресторанами и кухнями реализована через ассоциативную таблицу RestaurantCuisine, содержащую внешние ключи restaurant_id и cuisine_id. Это позволяет одному ресторану иметь несколько кухонь, а одну кухню — использовать во многих ресторанах. В API кухни ресторана возвращаются в составе объекта ресторана, что соответствует принципам REST.

7. Аутентификация и обработка ошибок

Защищённые эндпоинты требуют заголовок Authorization с Bearer-токеном. Токен доступа живёт один час, refresh-токен — семь дней.

Все ошибки возвращаются в едином формате с полями: code (строковый код), message (описание), status (HTTP-код). Основные типы: 400 (конфликт данных), 401 (нет авторизации), 403 (нет прав), 404 (ресурс не найден), 409 (конфликт состояния), 422 (невалидные данные), 500 (внутренняя ошибка).

8. Визуализация

Спецификация API оформлена в формате OpenAPI 3.0 и визуализирована в Swagger Editor. В документации API представлены все эндпоинты, их методы, параметры, структуры запросов и ответов, а также коды ошибок.

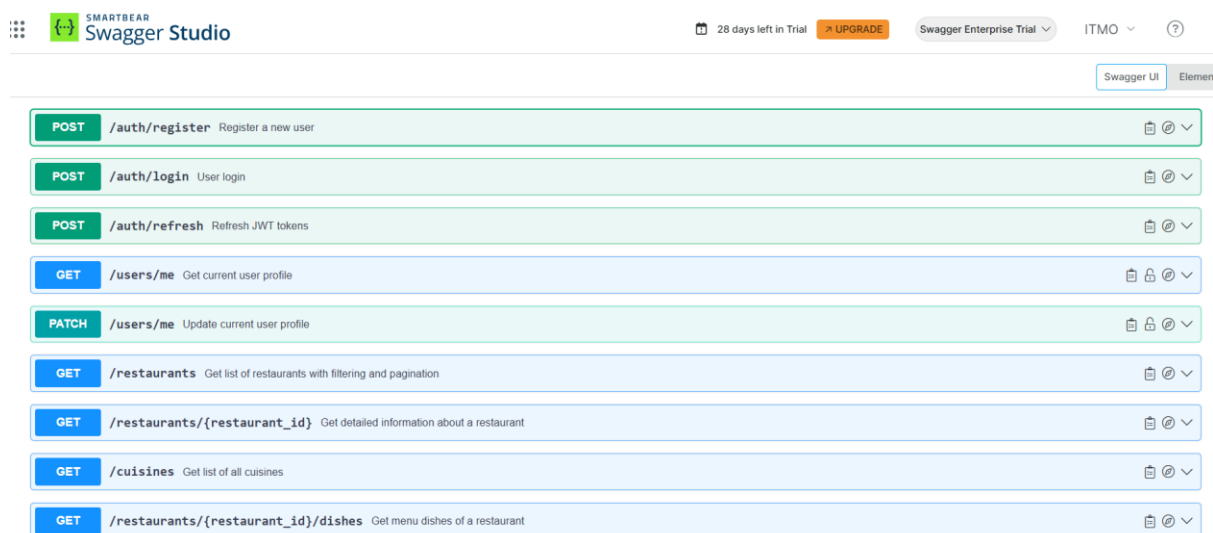


Рисунок 1 — Swagger

Вывод

В результате выполнения домашнего задания спроектирован REST API для сервиса бронирования столиков. Определены функциональные и нефункциональные требования, охватывающие весь необходимый функционал: от регистрации пользователей до управления бронированиями

и отзывами. Выбрана архитектура REST API с документированием в формате OpenAPI 3.0. Все эндпоинты описаны с указанием методов, параметров, структур запросов и ответов. Особое внимание уделено корректной реализации связи «многие-ко-многим» между ресторанами и кухнями через ассоциативную таблицу RestaurantCuisine. Продумана система аутентификации на JWT-токенах и единый формат обработки ошибок. Разработанная спецификация полностью соответствует ER-диаграмме из ДЗ1 и готова к реализации на Express.js с TypeORM.